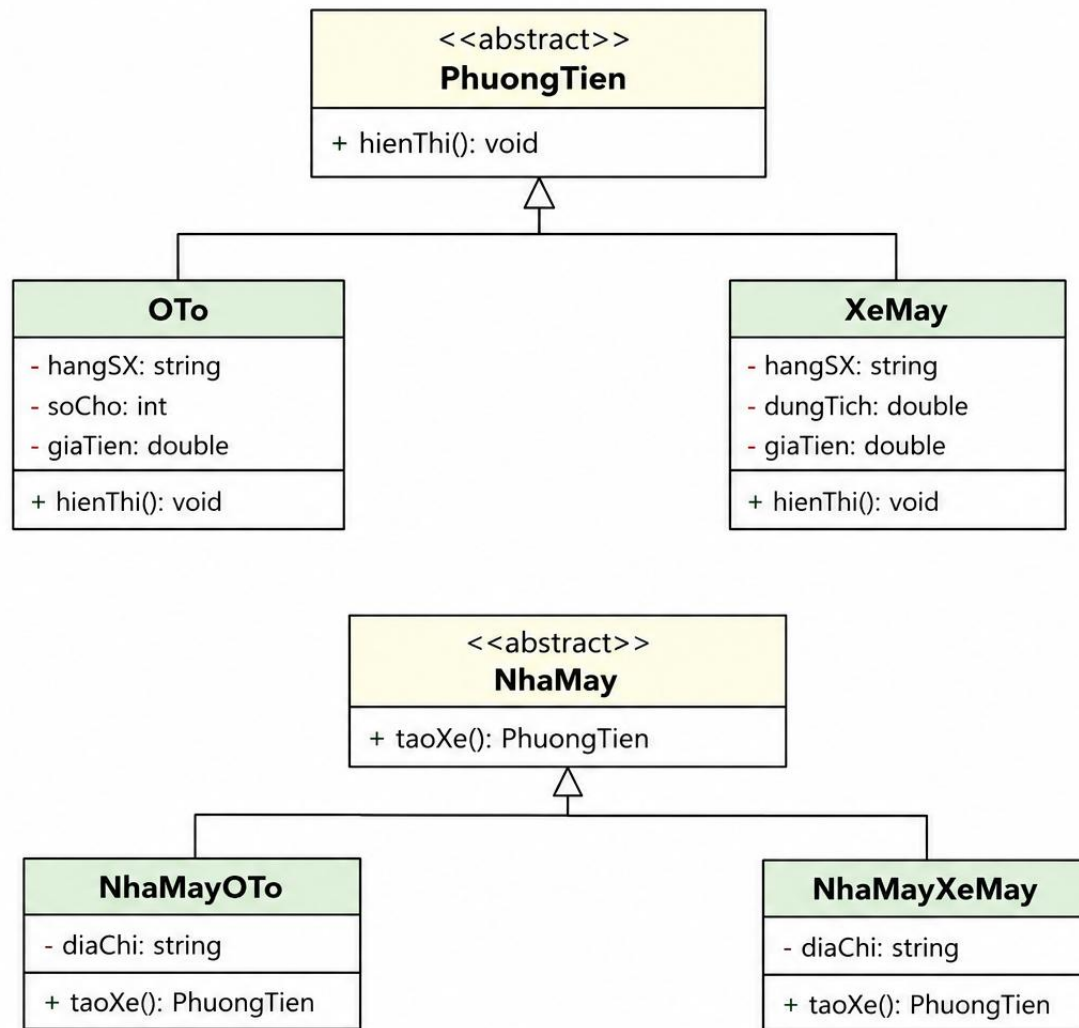


Khái niệm

Factory Method là mẫu thiết kế tạo đối tượng thông qua phương thức factory thay vì dùng new trực tiếp.

- Giúp dễ mở rộng
- Giảm phụ thuộc giữa các class



```

1 // Lop truu tuong
  7 usages
2 @ public abstract class PhuongTien {
    2 usages
3 i↓ abstract void hienThi();
4 }
5 // Lop con
  1 usage
6 class OTo extends PhuongTien {
    2 usages
7 i↑ void hienThi() {
8     | System.out.println("Day la O To");
9     | }
10 }
11
    1 usage
12 class XeMay extends PhuongTien {
    2 usages
13 i↑ void hienThi() {
14     | System.out.println("Day la Xe May");
15     | }
16 }

```

```

17 // Factory Method
  4 usages  2 inheritors
18 @ abstract class NhaMay {
    2 usages  2 implementations
19 i↓ abstract PhuongTien taoXe();
20 }
21
    1 usage
22 class NhaMayOTo extends NhaMay {
    2 usages
23 i↑ PhuongTien taoXe() {
24     | return new OTo();
25     | }
26 }
27
    1 usage
28 class NhaMayXeMay extends NhaMay {
    2 usages
29 i↑ PhuongTien taoXe() {
30     | return new XeMay();
31     | }
32 }

```

```
1
2 ▶ public class Main {
3 ▶     public static void main(String[] args) {
4         NhaMay nm1 = new NhaMay0To();
5         PhuongTien xe1 = nm1.taoXe();
6         xe1.hienThi();
7
8         NhaMay nm2 = new NhaMayXeMay();
9         PhuongTien xe2 = nm2.taoXe();
10        xe2.hienThi();
11    }
12 }
```

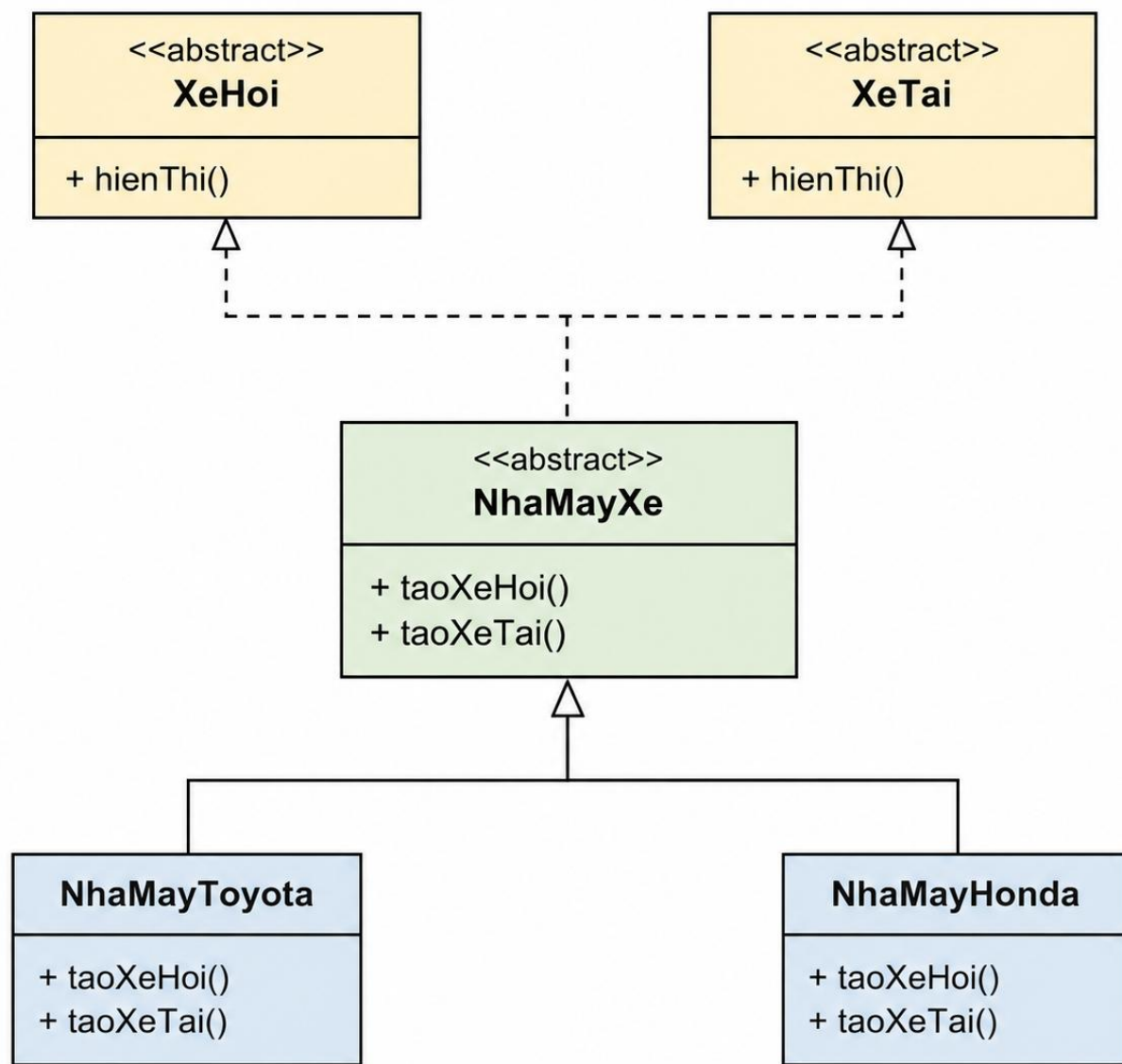
Abstract Factory

Khái niệm

Abstract Factory là mẫu thiết kế tạo ra họ đối tượng liên quan với nhau.

Ví dụ:

- Hãng Toyota → tạo Ô tô + Xe tải Toyota
- Hãng Honda → tạo Ô tô + Xe tải Honda



```

1 // San pham 1
  6 usages  2 implementations
2  interface XeHoi {
    1 usage  2 implementations
3      void hienThi();
4 }
5
6 // San pham 2
  6 usages  2 implementations
7  interface XeTai {
    1 usage  2 implementations
8      void hienThi();
9 }
10
11 // Abstract Factory
   3 usages
12  interface NhaMayXe {
    1 usage
13      XeHoi taoXeHoi();
    1 usage
14      XeTai taoXeTai();
15 }

```

```

1 // Toyota
  1 usage
2  class XeHoiToyota implements XeHoi {
    1 usage
3       public void hienThi() {
4         System.out.println("Xe hoi Toyota");
5     }
6 }
7
   1 usage
8  class XeTaiToyota implements XeTai {
    1 usage
9       public void hienThi() {
10         System.out.println("Xe tai Toyota");
11     }
12 }
13
14 // Honda
   1 usage
15  class XeHoiHonda implements XeHoi {
    1 usage
16       public void hienThi() {
17         System.out.println("Xe hoi Honda");
18     }
19 }

```

```

21  class XeTaiHonda implements XeTai {
    1 usage
22  public void hienThi() {
    |      System.out.println("Xe tai Honda");
24  }
25  }
26
27  // Factory cu the
    1 usage
28  class NhaMayToyota implements NhaMayXe {
29
    1 usage
30  public XeHoi taoXeHoi() {
    |      return new XeHoiToyota();
31  }
32  }
33
    1 usage
34  public XeTai taoXeTai() {
    |      return new XeTaiToyota();
35  }
36  }
37  }
38

```

```

39  class NhaMayHonda implements NhaMayXe {
40
    1 usage
41  public XeHoi taoXeHoi() {
    |      return new XeHoiHonda();
42  }
43  }
44
    1 usage
45  public XeTai taoXeTai() {
    |      return new XeTaiHonda();
46  }
47  }
48  }

```

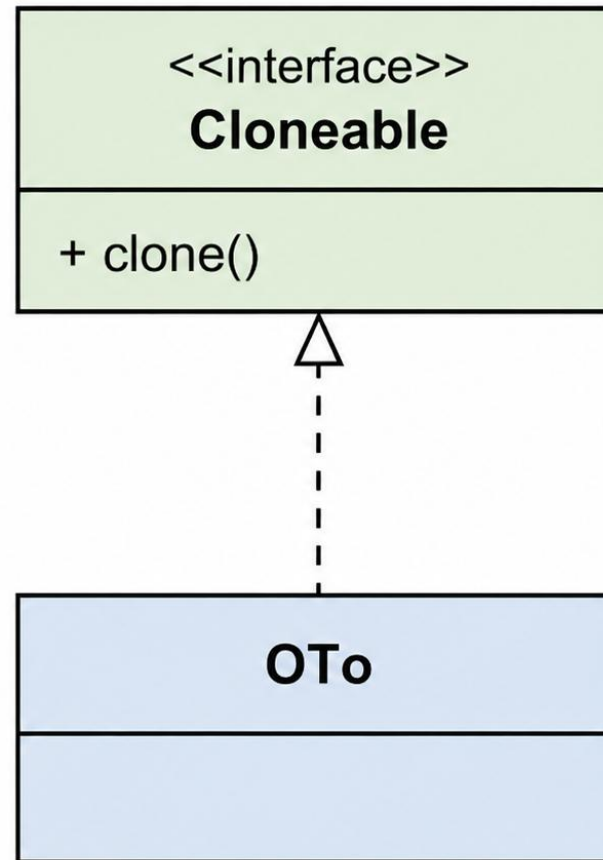


```
2  ▶ public class Main {  
3  ▶     public static void main(String[] args) {  
4      NhaMayXe toyota = new NhaMayToyota();  
5  
6      XeHoi xh = toyota.taoXeHoi();  
7      XeTai xt = toyota.taoXeTai();  
8  
9      xh.hienThi();  
10     xt.hienThi();  
11     }  
12 }
```

Khái niệm

Prototype Pattern dùng để sao chép đối tượng thay vì tạo mới.

- Tăng tốc tạo object
- Hữu ích khi object phức tạp

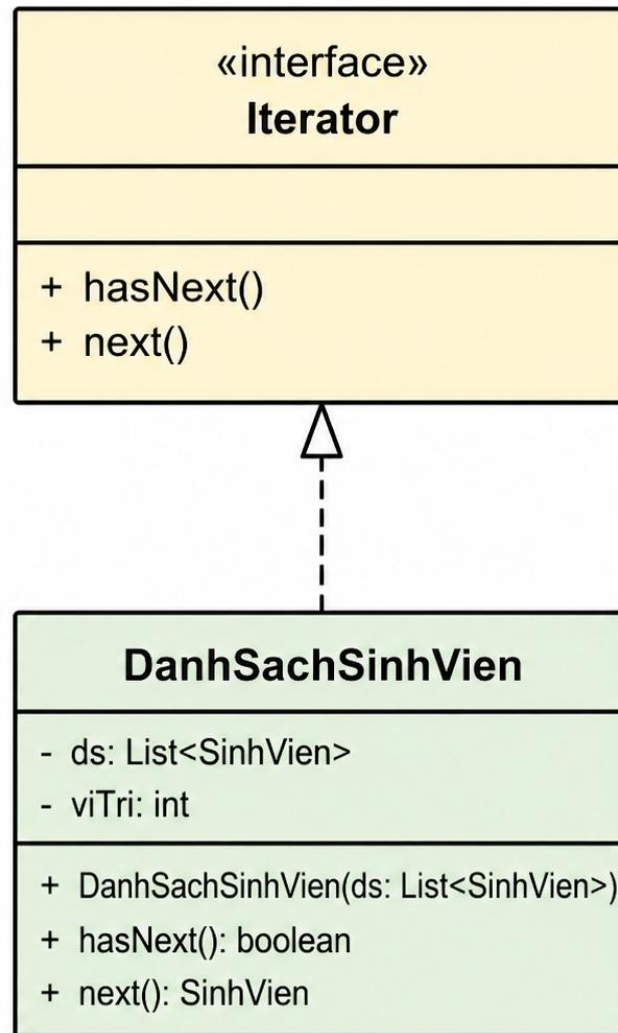


```
4 usages
1  class OTo implements Cloneable {
2
3      2 usages
4      String tenXe;
5      2 usages
6      int gia;
7
8      1 usage
9      public OTo(String tenXe, int gia) {
10         this.tenXe = tenXe;
11         this.gia = gia;
12     }
13
14     2 usages
15     public void hienThi() {
16         System.out.println(tenXe + " - " + gia);
17     }
18
19     protected Object clone() throws CloneNotSupportedException {
20         return super.clone();
21     }
22 }
```

```
1
2 ▶ public class Main {
3 ▶   public static void main(String[] args) throws CloneNotSupportedException {
4
5       Oto xe1 = new Oto( tenXe: "Toyota", gia: 500);
6
7       Oto xe2 = (Oto) xe1.clone();
8
9       xe1.hienThi();
10      xe2.hienThi();
11  }
12  }
13
```

Khái niệm

Iterator Pattern cho phép duyệt tập hợp phần tử mà không cần biết cấu trúc bên trong.



```

1  class SinhVien {
    2  usages
2    String ten;
3
    3  usages
4    public SinhVien(String ten) {
5        this.ten = ten;
6    }
7  }
    3  usages
8  class DanhSachSinhVien {
9
    2  usages
10     SinhVien[] ds = {
11         new SinhVien( ten: "An"),
12         new SinhVien( ten: "Binh"),
13         new SinhVien( ten: "Cuong")
14     };
    1  usage
15     public Iterator layIterator() {
16         return new Iterator();
17     }

```

```

18     class Iterator {
19
    2  usages
20         int viTri = 0;
21
    1  usage
22         boolean hasNext() {
23             return viTri < ds.length;
24         }
25
    1  usage
26         SinhVien next() {
27             return ds[viTri++];
28         }
29     }
30 }

```

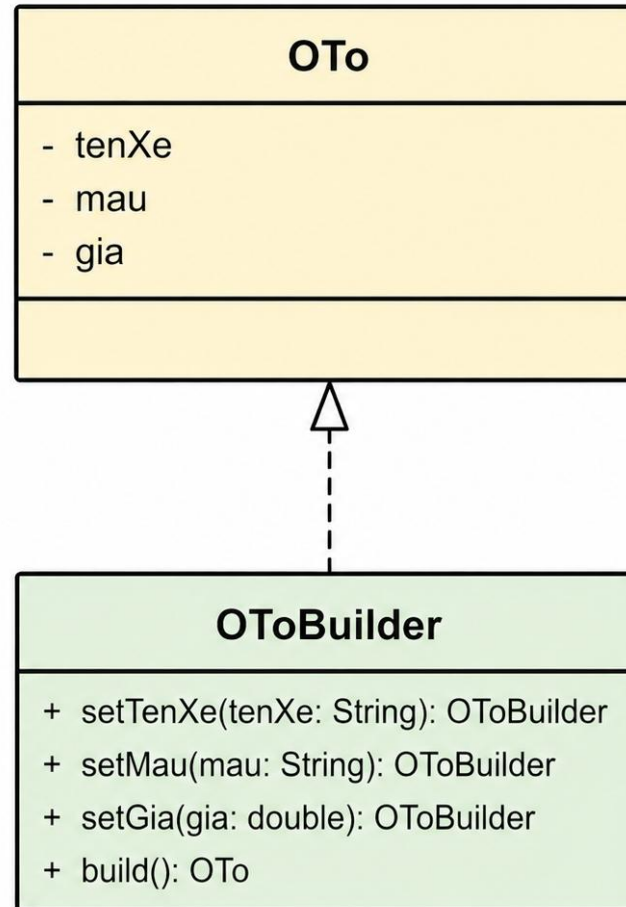


```
2  ▶ public class Main {  
3  ▶     public static void main(String[] args) throws CloneNotSupportedException {  
4  
5         DanhSachSinhVien ds = new DanhSachSinhVien();  
6  
7         DanhSachSinhVien.Iterator it = ds.layIterator();  
8  
9         while (it.hasNext()) {  
10             SinhVien sv = it.next();  
11             System.out.println(sv.ten);  
12         }  
13     }  
14 }
```

Khái niệm

Builder Pattern dùng để tạo đối tượng phức tạp từng bước.

- Dễ đọc
- Tránh constructor quá nhiều tham số



```

1  class OTo {
2
3      2 usages
      String tenXe;
4      2 usages
      String mau;
5      2 usages
      int gia;
6
7      1 usage
      public OTo(String tenXe, String mau, int gia) {
8          this.tenXe = tenXe;
9          this.mau = mau;
10         this.gia = gia;
11     }
12     1 usage
      public void hienThi() {
13         System.out.println(tenXe + " - " + mau + " - " + gia);
14     }
15 }

```

```

16 class OToBuilder {
17
18     2 usages
      String tenXe;
19     2 usages
      String mau;
20     2 usages
      int gia;
21     1 usage
      public OToBuilder setTenXe(String tenXe) {
22         this.tenXe = tenXe;
23         return this;
24     }
25     1 usage
      public OToBuilder setMau(String mau) {
26         this.mau = mau;
27         return this;
28     }
29
30     1 usage
      public OToBuilder setGia(int gia) {
31         this.gia = gia;
32         return this;
33     }
34     1 usage
      public OTo build() {
35         return new OTo(tenXe, mau, gia);
36     }
37 }

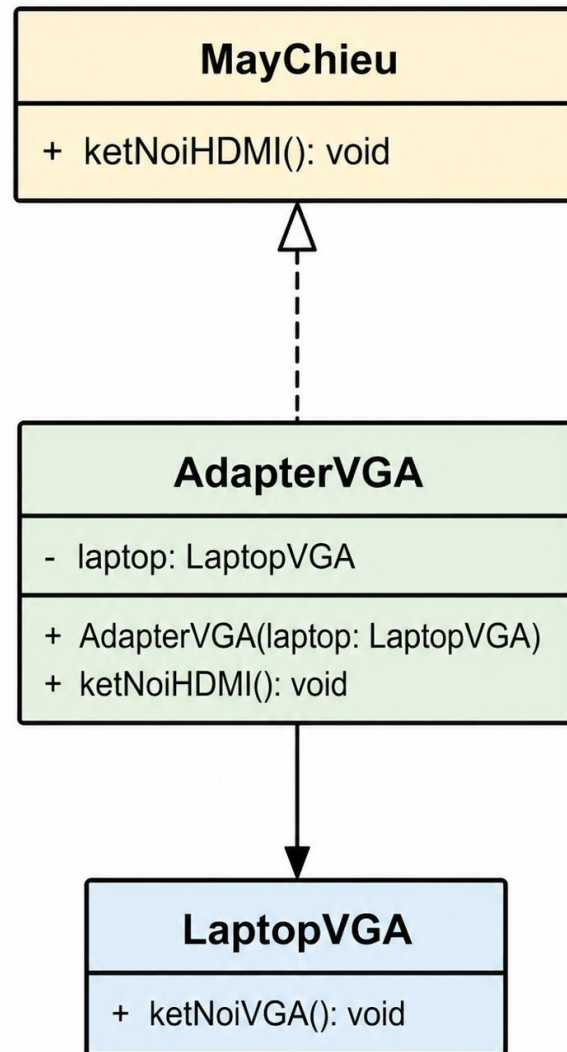
```

```
2  ▶ public class Main {  
3  ▶  ◯ public static void main(String[] args) throws CloneNotSupportedException {  
4        
5          OTo xe = new OToBuilder()  
6              .setTenXe("Toyota")  
7              .setMau("Do")  
8              .setGia(500)  
9              .build();  
10   
11      xe.hienThi();  
12  }  
13 }
```

Adapter

Khái niệm

Adapter Pattern giúp hai interface không tương thích làm việc với nhau.



```

1 // Target
  2 usages 1 implementation
2 ⓘ interface HDMI {
    1 usage 1 implementation
3 ⓘ void ketNoiHDMI();
4 }

```

```

6 // Adaptee
  4 usages
7 class LaptopVGA {
8
    1 usage
9     public void ketNoiVGA() {
10         System.out.println("Ket noi cong VGA");
11     }
12 }
13
14 // Adapter
  1 usage
15 class AdapterVGA implements HDMI {
16
    2 usages
17     LaptopVGA laptop;
18
    1 usage
19     public AdapterVGA(LaptopVGA laptop) {
20         this.laptop = laptop;
21     }
22
    1 usage
23 ⓘ public void ketNoiHDMI() {
24     laptop.ketNoiVGA();
25 }
26 }

```



```
1
2 ▶ public class Main {
3 ▶     public static void main(String[] args) throws CloneNotSupportedException {
4
5         LaptopVGA laptop = new LaptopVGA();
6
7         HDMI adapter = new AdapterVGA(laptop);
8
9         adapter.ketNoiHDMI();
10    }
11 }
```